

Getting started with Amazon Elastic Kubernetes Service (Amazon EKS)

AWS Prescriptive Guidance

Amazon Elastic Kubernetes Service (Amazon EKS) is a managed Kubernetes service that makes it easy to run Kubernetes on AWS without needing to install, operate, and maintain your own Kubernetes control plane or nodes. This guide covers EKS architecture, node groups, Fargate profiles, networking with VPC CNI, storage, security, cluster autoscaling, observability, GitOps patterns, and production best practices.

Authors: Tomasz Kowalski and Amara Diallo, Amazon Web Services (AWS)

Date: February 2026

Table of Contents

- [Introduction](#)
- [EKS Architecture](#)
- [Compute Options](#)
- [Networking with VPC CNI](#)
- [Storage on EKS](#)
- [IAM Integration and Security](#)
- [Cluster Autoscaling](#)
- [Observability](#)
- [GitOps on EKS](#)
- [Best Practices](#)
- [FAQ](#)

Introduction

Amazon EKS runs upstream Kubernetes and is certified Kubernetes conformant, so applications managed by EKS are fully compatible with any standard Kubernetes environment. EKS manages the Kubernetes control plane across multiple AWS Availability Zones, automatically detecting and replacing unhealthy control plane instances. EKS also integrates with AWS services such as ECR, IAM, VPC, ELB, CloudWatch, and CloudTrail.

Kubernetes on EKS supports the same Kubernetes API, kubectl tooling, and ecosystem tooling as any other Kubernetes distribution. EKS runs the same version of Kubernetes that you can find on GitHub, with backported security patches. EKS supports standard Kubernetes add-ons including CoreDNS, kube-proxy, and the VPC CNI plugin, as well as additional managed add-ons for observability, networking, and storage.

EKS Architecture

An EKS cluster consists of two planes: the control plane and the data plane. The EKS control plane — including the Kubernetes API server, etcd, scheduler, and controller manager — is managed by AWS. The control plane runs in AWS-managed accounts, dedicated to your cluster, across multiple Availability Zones. AWS manages patching, upgrading, and scaling the control plane automatically.

The data plane consists of EC2 worker nodes or AWS Fargate pods that run your containerized workloads. Worker nodes are registered with the control plane via the Kubernetes Node API. You are responsible for managing the data plane, though EKS Managed Node Groups simplify this significantly by automating node provisioning, registration, and lifecycle management.

Compute Options

Option	Management	Customization	Pricing Model	Best For
Managed Node Groups	Semi-managed	High (AMI, instance type)	EC2 pricing	General-purpose workloads
Self-managed Nodes	Manual	Full	EC2 pricing	Custom AMIs, specific requirements
AWS Fargate	Fully managed	Limited	Per pod vCPU/memory	Batch jobs, dev/test, microservices
Karpenter (auto-provisioning)	Semi-managed	High	EC2 pricing	Dynamic, cost-optimized scaling

Karpenter

Karpenter is an open-source, flexible, high-performance Kubernetes cluster autoscaler built for AWS. Unlike the Cluster Autoscaler which scales node groups, Karpenter directly provisions EC2 instances based on the pending pod specifications. Karpenter can provision nodes from a wide range of instance types, availability zones, and purchase options (On-Demand, Spot, Reserved) to minimize cost while satisfying scheduling constraints. Karpenter can consolidate workloads and terminate underutilized nodes, further reducing costs.

Networking with VPC CNI

The Amazon VPC CNI plugin (aws-vpc-cni-k8s) integrates Kubernetes networking with the AWS VPC networking model. Unlike overlay networks used by other CNI plugins, VPC CNI assigns VPC IP addresses directly to pods. Each pod receives a real VPC IP address from the subnet of the node it runs on, enabling native VPC routing, security group policies, and network observability without encapsulation overhead.

Security Groups for Pods is a VPC CNI feature that allows you to assign AWS Security Groups to Kubernetes pods, enabling network-level segmentation at the pod level. This is particularly useful for enforcing network policies for pods that access AWS services or on-premises resources over Direct Connect or VPN.

Note: IP address exhaustion is a common challenge with VPC CNI. Use prefix delegation to assign /28 IP address prefixes to nodes, significantly increasing the maximum pod density per node.

Storage on EKS

EKS supports persistent storage through Container Storage Interface (CSI) drivers. The Amazon EBS CSI driver enables dynamic provisioning of Amazon EBS volumes as Kubernetes Persistent Volumes (PVs). The Amazon EFS CSI driver enables ReadWriteMany (RWX) access to shared file storage across multiple pods and nodes. The Amazon FSx for Lustre CSI driver enables high-performance file storage for machine learning and HPC workloads.

IAM Integration and Security

IAM Roles for Service Accounts (IRSA) enables Kubernetes service accounts to assume IAM roles using the Kubernetes OIDC federation. This allows pods to have fine-grained AWS permissions without using node-level IAM instance profiles, following the principle of least privilege. When a pod authenticates to an AWS service, it assumes the IAM role associated with its service account via a projected service account token.

EKS Pod Identity is a newer mechanism that further simplifies IRSA configuration by eliminating the need to configure OIDC identity providers in IAM. Pod Identity uses a dedicated EKS service to handle token exchange, simplifying the setup and improving security.

Cluster Autoscaling

EKS supports horizontal scaling through the Kubernetes Horizontal Pod Autoscaler (HPA), which automatically scales the number of pod replicas based on CPU/memory utilization or custom metrics from Amazon CloudWatch or Prometheus. The Vertical Pod Autoscaler (VPA) recommends or automatically adjusts CPU and memory requests for containers. At the node level, Karpenter or the Cluster Autoscaler (CAS) provision and terminate nodes based on pending pod requirements.

Observability

EKS integrates with Amazon CloudWatch Container Insights, which collects, aggregates, and summarizes metrics and logs from containerized applications and microservices. Container Insights provides automated dashboards for cluster, node, pod, and service metrics. EKS control plane logs (API server, audit, authenticator, controller manager, scheduler) can be enabled and sent to CloudWatch Logs for compliance and troubleshooting.

For observability-as-code, AWS Managed Grafana and AWS Managed Service for Prometheus (AMP) provide fully managed Prometheus-compatible metrics storage and Grafana dashboards for Kubernetes clusters. Amazon OpenSearch Service can be used for log analytics and visualization using the popular EFK (Elasticsearch/OpenSearch, Fluentd, Kibana) stack.

Best Practices

- Use Managed Node Groups: Managed Node Groups automate node provisioning, updates, and termination, reducing operational overhead significantly.
- Implement IRSA for pod-level IAM permissions: Assign minimum-privilege IAM roles to Kubernetes service accounts rather than using broad node-level instance profiles.
- Use Karpenter for cost-optimized autoscaling: Karpenter provides faster, more flexible, and more cost-efficient node provisioning compared to the Cluster Autoscaler.
- Separate control and data planes across AZs: Ensure worker nodes span at least three Availability Zones for high availability of your workloads.
- Implement Pod Disruption Budgets (PDBs): Use PDBs to prevent disruption from voluntary events such as node draining during cluster upgrades.
- Keep clusters up to date: EKS supports four minor Kubernetes versions. Clusters on end-of-support versions are automatically upgraded to the next supported version.

- Use namespaces for multi-tenancy: Separate workloads into namespaces with appropriate RBAC and resource quotas to prevent resource contention.
- Enable private endpoint access: For production clusters, disable public endpoint access or restrict it to specific CIDR ranges, and enable private endpoint access.

Frequently Asked Questions

Q: What Kubernetes versions does EKS support?

A: EKS supports the four most recent Kubernetes minor versions. AWS releases support for new Kubernetes versions approximately one to two months after the upstream release and provides at least 14 months of support per minor version.

Q: How do I upgrade my EKS cluster?

A: EKS supports in-place cluster upgrades. First upgrade the control plane, then update managed node groups or re-provision self-managed nodes. Use the `eksctl` or Terraform to manage upgrades. Always test upgrades in a non-production cluster first.